

DOKUMENTACJA FUNKCJONALNOŚCI

Jedno źródło prawdy czy wiele plików?

Porównanie dwóch podejść do przekazywania specyfikacji od analityków do developerów — determinizm, wygoda, ryzyko utraty danych.



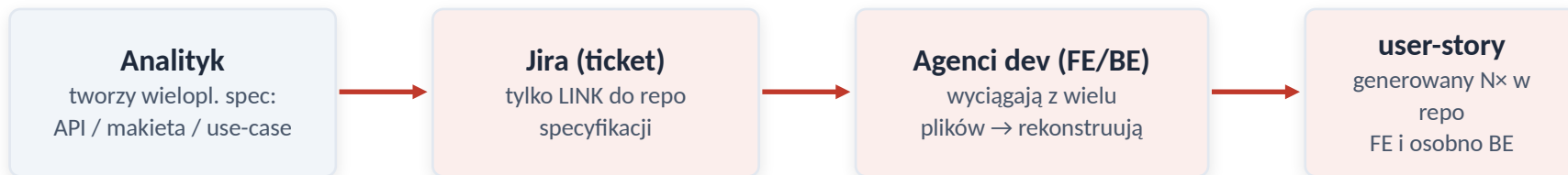
Podejście 1 — wieloplikowe, link w Jirze



Podejście 2 — konsolidacja, jedno źródło

Dwa przepływy tej samej specyfikacji

PODEJŚCIE 1



PODEJŚCIE 2



Różnica nie leży w formacie specyfikacji — leży w tym, KTO i ILE RAZY ją scala oraz GDZIE żyje wersja, na której pracują developerzy.



PODEJŚCIE 1 • CZEGO SIĘ SPODZIEWAĆ

Zagrożenia: niedeterminizm rozproszony



Rozjazd FE ↔ BE Agregacja wieloplikowa biegnie osobno dla FE i BE. Z tej samej spec powstają DWA różne user-story → różne kontrakty API, ciche niespójności wykrywane dopiero na integracji.



Niepowtarzalność przebiegów Każde ponowne uruchomienie agenta może dać inny wynik (kolejność plików, interpretacja LLM). Brak zamrożonego artefaktu = brak referencji „co właściwie zaimplementowano”.



Gubienie kontekstu między plikami Reguła z pliku API + warunek z use-case + element z makiety bywają rozdzielone; agent łączy je heurystycznie i potrafi pominąć zależność rozłożoną na 2-3 pliki.



Duplikacja pracy i pipeline'ów Każdy zespół buduje i utrzymuje własną ekstrakcję. Zmiana formatu spec = naprawa w wielu miejscach. Koszt utrzymania rośnie z liczbą zespołów.



PODEJŚCIE 2 • CZEGO SIĘ SPODZIEWAĆ

Zagrożenia: ryzyko skupione w jednym punkcie



Pojedynczy punkt awarii (SPOF) Jeśli agent-konsolidator spłaszczy lub zgubi treść, błąd propaguje się do WSZYSTKICH developerów naraz — a źródłem prawdy jest już zubożony tekst, więc trudniej to wykryć.



Desynchronizacja Jira ↔ repo Treść w Jirze to migawka. Gdy specyfikacja w repo się zmieni, ticket pozostaje stary. Bez wskazania wersji nie wiadomo, czy user-story odpowiada aktualnej spec.



Ręczna edycja w Jirze Pole tekstowe kusi do „szybkiej poprawki”. Każda ręczna zmiana rozjeżdża treść z repo i niszczy powtarzalność (regeneracja ją nadpisze lub powstanie konflikt).



Słaba audytowalność treści Jira nie wersjonuje treści jak git. Trudniej o diff, review i odpowiedź „co dokładnie zmieniło się między wersjami zadania”.

Plusy i minusy — strona po stronie

Podejście 1 — wieloplikowe

Z A L E T Y

- ✓ Źródło (repo) pełne i audytowalne — git daje diff/review/wersje
- ✓ Analityk pisze natywnie, bez kroku konsolidacji
- ✓ Strata kontekstu jest „naprawialna” — wystarczy ponowny przebieg

W A D Y

- ✗ Niedeterminizm: różne wyniki FE vs BE i przebieg po przebiegu
- ✗ Duplikacja pipeline'ów ekstrakcji w każdym zespole
- ✗ Ciche rozjazdy kontraktów wykrywane późno

Podejście 2 — jedno źródło

Z A L E T Y

- ✓ Determinizm: jedna agregacja, identyczny tekst dla wszystkich
- ✓ Zero ekstrakcji po stronie dev — najniższa wariancja
- ✓ Spójność FE/BE; błąd (jeśli jest) wspólny i wykrywalny

W A D Y

- ✗ SPOF w kroku konsolidacji — błąd dotyka wszystkich
- ✗ Ryzyko desync Jira ↔ repo i ręcznych edycji
- ✗ Słabsza audytowalność treści niż w gicie

Determinizm · Wygoda · Utrata danych

Kryterium	Podjęcie 1 — wieloplikowe	Podjęcie 2 — jedno źródło
Determinizm	 Niski — N agregacji, rozjazd FE/BE	 Wysoki — 1 agregacja, wspólny wynik
Wygoda — analityk	 Wyższa — pisze natywnie, bez scalania	 Niższa — dodatkowy krok konsolidacji
Wygoda — developer	 Niższa — własny pipeline ekstrakcji	 Wyższa — jedno źródło, zero pracy
Utrata danych	 Rozproszona, naprawialna (źródło pełne)	 Skupiona w 1 kroku, propaguje się
Audytowalność	 Wysoka — git na specyfikacji	 Niższa — treść w polu Jiry
Spójność zespołu	 Słaba — możliwe rozbieżne kontrakty	 Mocna — wszyscy czytają to samo



PRACA ZESPOŁOWA

Kilku analityków na jednym repo specyfikacji

Repo to git — i to git rozwiązuje większość problemów współbieżności. Zasady, które warto wymusić:



Izolacja przez strukturę

Jeden folder na funkcjonalność (np. /features/<klucz-jira>/). Każdy analityk pracuje w swoim katalogu → konflikty merge praktycznie znikają.



Branch + PR per funkcjonalność

Zmiany przez gałąź i pull request, nie push na main. Review między analitykami i czytelna historia kto/co/kiedy.



Konwencja nazewnictwa = klucz Jira

Folder/branch nazwany kluczem ticketu wiąże spec ze zadaniem 1:1 i eliminuje zgadywanie „która spec do którego ticketu”.



Zamrożenie wersji do wydania

Po akceptacji: tag / commit SHA. To ON jest referencją dla developerów — nie „latest”, który ktoś może w międzyczasie zmienić.

Zmiana w istniejącej funkcjonalności

Reguła decyzyjna zależy od JEDNEGO pytania: czy funkcjonalność jest już zaimplementowana?



Jeszcze NIE zaimplementowane

Tryb: REGENERACJA całej Story.

- Zmiany wchodzą jako nowy przebieg agenta — nadpisują dotychczasową treść w całości.
- Nikt jeszcze nie pracuje na starej wersji, więc nadpisanie nie niszczy żadnej pracy.
- Efekt: Story zawsze odzwierciedla aktualny, pełny zakres.



JUŻ zaimplementowane

Tryb: OSOBNE zadanie — tylko zakres zmian.

- Analityk wrzuca wyłącznie poprawki (delta), nie regeneruje całej Story.
- Pierwotna Story zostaje jako zapis tego, co faktycznie wdrożono — bez nadpisywania.
- Nowy ticket niesie czytelny, wąski diff: dev widzi dokładnie, co się zmienia.

Zasada: regeneracja tylko przed implementacją. Po wdrożeniu zmiana = osobne zadanie z samym zakresem poprawek — nigdy nadpisanie zaimplementowanej Story.



REKOMENDACJA

Wybierz determinizm — ale zabezpiecz jego słaby punkt

Podejście 2 wygrywa na najważniejszym kryterium — spójności FE/BE. Jego jedyne realne ryzyko (SPOF + desync) neutralizuje się dyscypliną wersjonowania:



Zamrożona wersja w tickecie

W treści Jiry obok spec umieść commit SHA / tag spec źródłowej. Odtwarza audytowalność, której Jira sama nie ma.



Manifest źródeł

Lista plików + ich SHA, które weszły do konsolidacji. Widać, co dokładnie zostało scalone — łatwiej wykryć braki.



Regeneracja zależna od fazy

Przed implementacją: pełna regeneracja. Po wdrożeniu: osobny ticket tylko z deltą — bez nadpisywania Story.



Repo jako źródło prawdy

Skonsolidowany artefakt także ładuje w gicie (jeśli to możliwe). Jira to widok, nie magazyn.